

UC: In Docker we have **run multiple containers, within the same host**. When we need HA, we need more machines to host containers.

Problem Statement:

Work.. Work.. Work..



How to Organize??



Why Manager??

- ❖ To Assign Work
- ❖ To get New Projects
- ❖ Leave Management
- ❖ Work Management
- ❖ Promotions
- ❖ Adding More Members
- ❖ Monitoring



How to Manage More Containers???



Cluster: Group of servers

If you deploy your application across 100 of servers in cluster then how you manage your applications?

Problem Statement:

- How containers communicate to each other?
- How container communicate to each other across multiple nodes?
- How implement Autoscaling?
- How to manage/monitor cluster (group of servers)?
- High Availability?
- Security?

Container-based application brings many challenges to IT & DevOps teams

- Managing the underlying infrastructure on which these apps are running.
- Managing the actual applications, which is especially challenging in upstream environments like PROD.

Once application is deployed, there are several requirements for managing them

- High Availability- distributed hosts/regions
- Backups
- Monitoring & Alerts
- Auto-Scaling
- Continuous Delivery for Enterprise Apps
- Service Discovery
- Logging
- Integration with Existing Services

● Auditing

Solution: Container Orchestration tools (docker swarm, K8s)

We need container orchestration, so that our containers are highly available

Why Container Orchestration?

Hosting container-based applications in different environment brings many challenges to Infrastructure Teams & DevOps teams such as...

- Which NODE suits the Container Requirement ?
- Maintain the Desired State ?
- How to Share the Load (User Requests) ?
- Scalability ?
- Auto Healing ?
- Auto Scaling ?
- ZERO Downtime Upgrades/ Rollbacks
- Containers & NODEs communication
- Security-- RBAC and Certificates



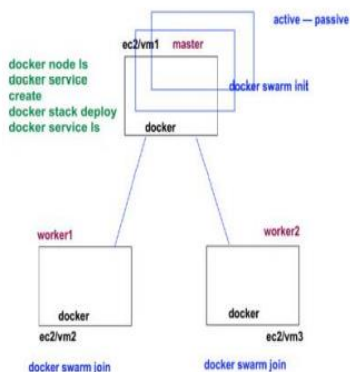
What is the difference between Docker Swarm and K8s?

in any container orchestration cluster —
cluster ? group of vms / servers / nodes

1 manager (can be multiple managers)
1 worker node (can be multiple worker nodes)

docker swarm — cluster of docker nodes

cluster — group of vms /server/nodes/machines
docker node ? a vm / server /node/machine where you install docker



kubernetes — container orchestrator
is just container management tools

AUTOSCALING
RBAC — role based access control

by default in kube cluster there is no container networking drivers /
channel for containers communication across multiple vms

we must install a networking driver as part of cluster init / setup

— flannel / canal / calico / weave / kube router

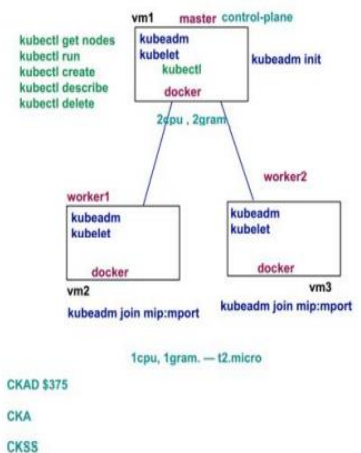
we can install only any one of networking solutions

kubernetes
— is written go lang
— init release was in 2014
— google had a product called BORG as orchestrator in house
— in 2014 google donated the BORG to open source community &
renamed as kubernetes
— from 2014 is managed by CNCF — open source community
— docker / containerization first released in 2013 — go lang

kubeadm: is a utility
— helps to init the master node
— helps to join worker to master node

kubelet: is an agent process
— that runs on every node in the cluster
— is responsible for master & worker node communication
— this process initializes when we run kubeadm join

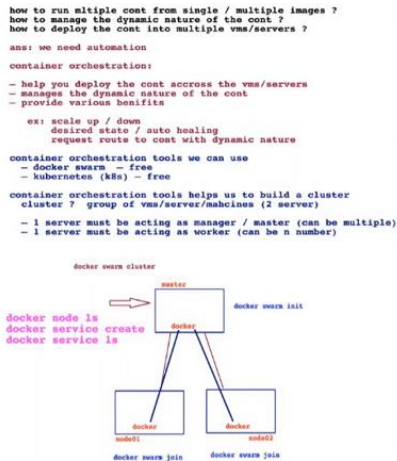
containerization tools
docker / containerd / rocket / ixc



CKAD \$375

CKA

CKSS



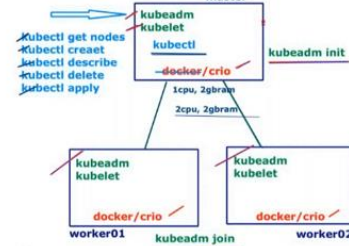
kubernetes(k8s):
 - is just container orchestration tool / cont mgmt tool
 - kube needs one of the containerization tool to be installed on the vms

docker / crio / containerd / rocket

- kube is written using GoLang
 - kube is first release was in 2014

AutoScaling
 RBAC - role based access control
 SSL -

control-plane



kubeadm:
 - is a tool helps to init the kube master process / node
 - is a tool helps join the worker to master node

master process on the master vm communicates with agent process on the worker vm

both are the container orchestrator, group of hosts to create a cluster.

K8s support different containerization techniques

- docker
- rocket
- lxc
- containerd

Flexible in containerization techniques

Flexible in networking solution (**Weave**, Calico, Flannel)

heavier than docker swarm (2CPU,4GB)

- autoscaling
- scheduling
- ingress (Network Load Balancing)
- package manager (helm)
- network policy
- RBAC

free -m

K8s sits on top of docker to manage the containers, while **Docker provides run time environment to run the containers**. K8S is container management tool.

How do we setup K8s Cluster?

There are two ways to work with k8s?

K8s as a service (KAS) - Cluster is created by cloud vendor and handover to you.

All cloud provider provides k8s as a service. All responsibilities of the utilities handled by cloud provider. As a user we have to demand only for number of servers in cluster. Charged only for servers in cluster not for service.

EKS provided by Amazon, **Google Kubernetes Engine** by Google cloud.

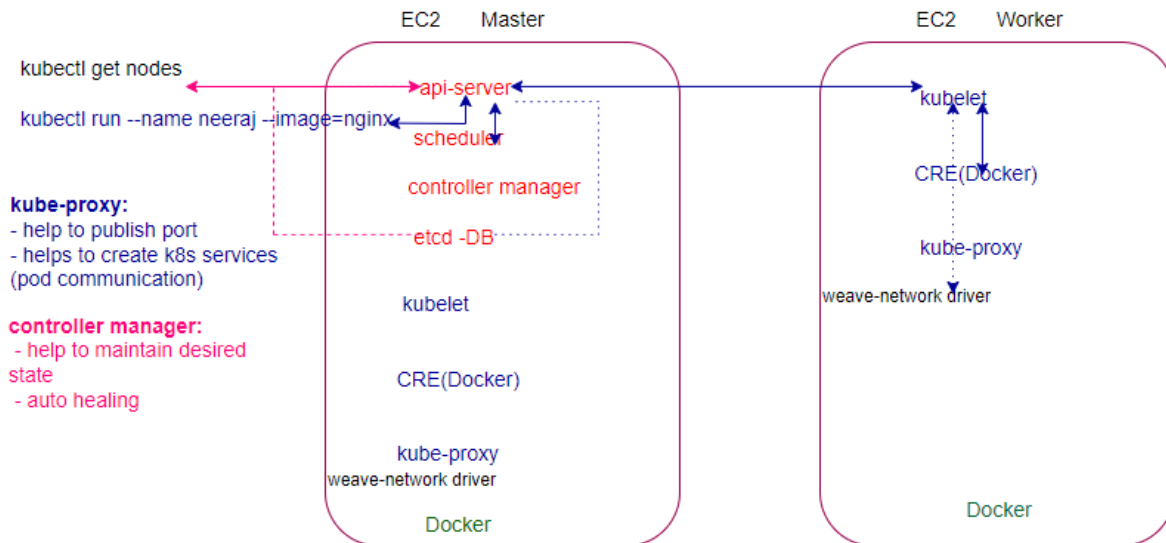
Kubernetes managed by cloud provider.

EKS? Cluster is created by AWS and handover to you.

Self-hosted

self-created cluster using kubeadm utility as a pod.

Components:



api-server: Single point of contact

scheduler- assign node

controller manager- desired state, self- healing

etcd-metadata (key-value database store cluster information)

kubectl get nodes

request->api-server->etcd

kubectl run --name Neeraj --image=nginx

request->api-server->scheduler->api-server->kubelet->Docker& kube-proxy->kubelet->api-server->etcd

Kubernetes - k8s

CNI -- container network interface

is set of rules how containers should communicate across cluster members

kubernetes doesn't come up with any container communication channel by default.

we must install a network driver as part of cluster setup

there are different vendors who follow the CNI rules defined by kubernetes & develops the networking drivers

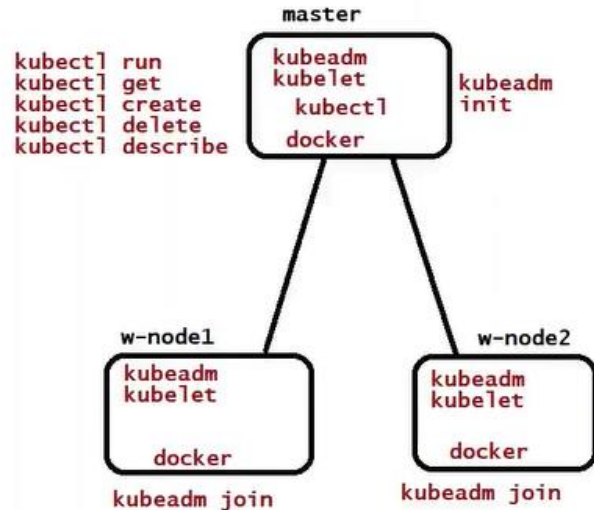
weave/flannel/calico/canal

RBAC -- Role Based Access Control

AutoScaling

containerization tools

docker
rocket
containerd
lxc
crio



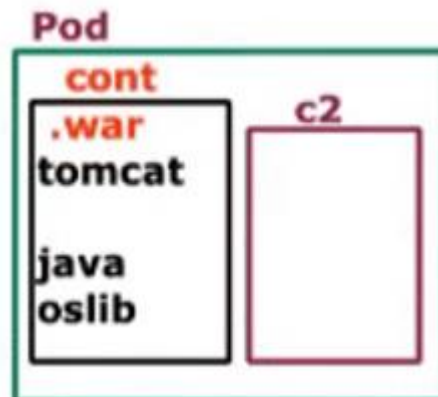
Kubeadm: is a utility helps us initialize cluster / upgrade / de init

kubelet: it is a kubernetes agent that is responsible for master & worker communication

kubectl: is client utility helps us interact master

Kubernetes Pods:

- Is an abstract layer that can create and manage a single/group of containers.
- Container runs inside the pod always
- Single pod==single Instance of an app



To achieve HA, must have multiple pods, not the multiple containers.

`kubectl run webapp --image=gcr.io/google-container/nginx: latest`

If node fail, will K8s take care?

No, Its Infrastructure related Issue, K8s take care only the containers.

kubelet: Is a command line utility (agent) sits on every node and responsible to report the status of the nodes, pods to master (cAdvisor -> Metrics Server).
Initiate the health check.

How Nodes Knows Manager is down?

It's not responsibility of node to know about master, will continue with present workload, but no new deployment will be allowed, and feature provided by master will not be available.

How to interact with K8s master?

kubectl: Is a command line utility installed only on master to communicate with nodes, client to interact with cluster to create, delete, edit the resources within the cluster.

How to Install K8s?

HARDWAY

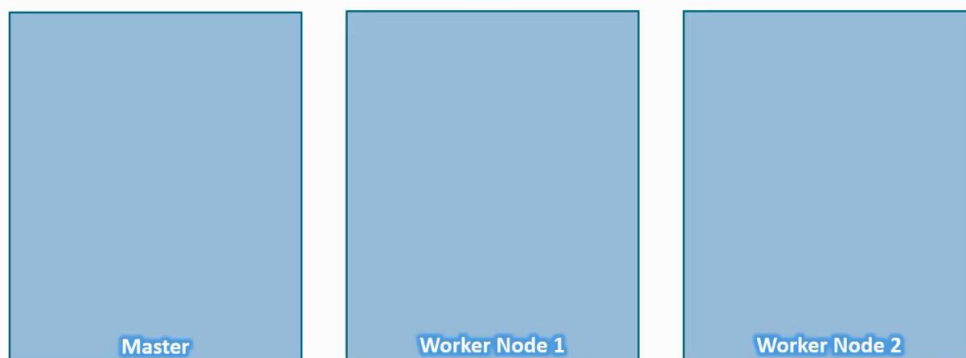
Kubeadm way?

Kubeadm is a utility takes responsibility to install all the components and establish a communication between the components.

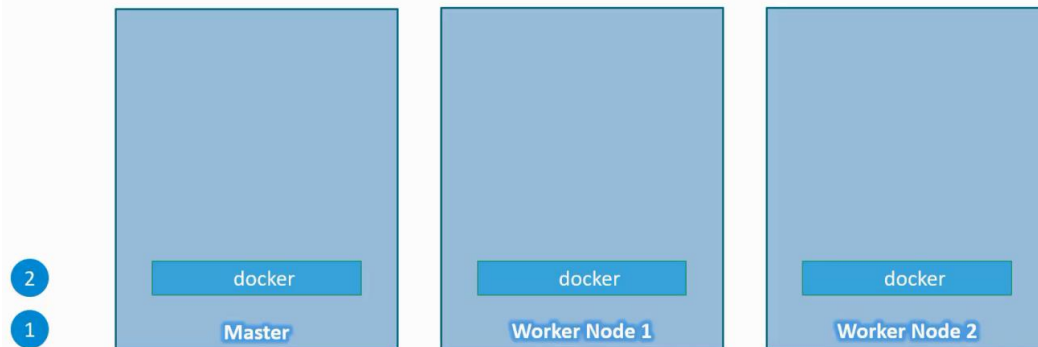
kubeadm will not install or manage **kubelet** or **kubectl** for you, so you will need to ensure they match the version of the Kubernetes control plane you want kubeadm to install for you.

Steps

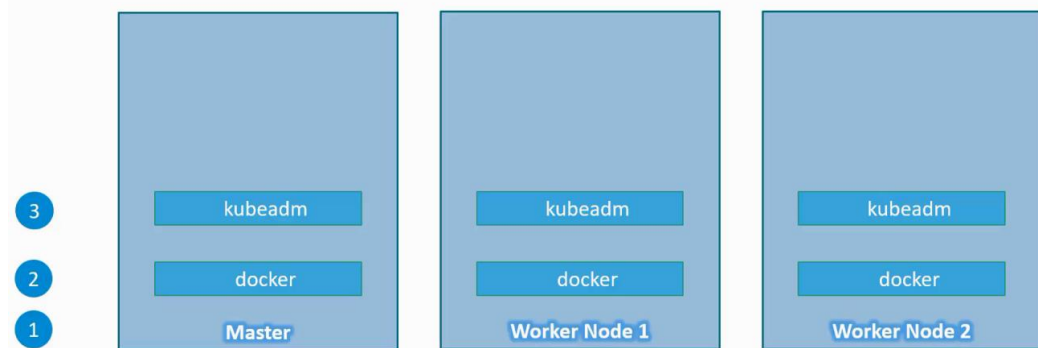
1



Steps

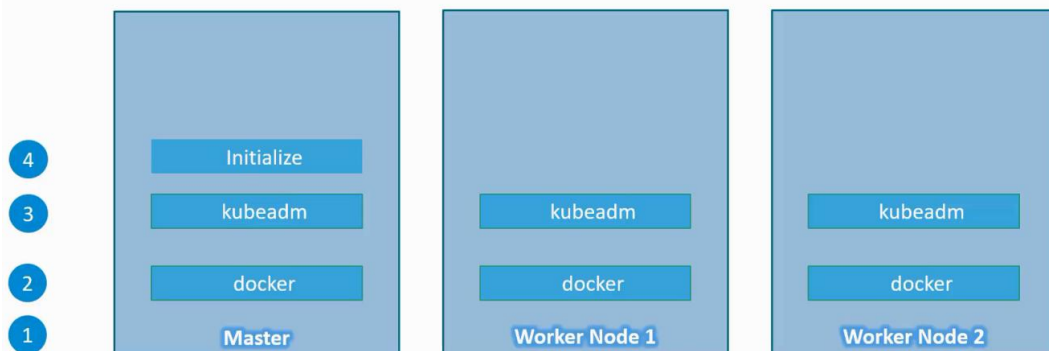


Steps



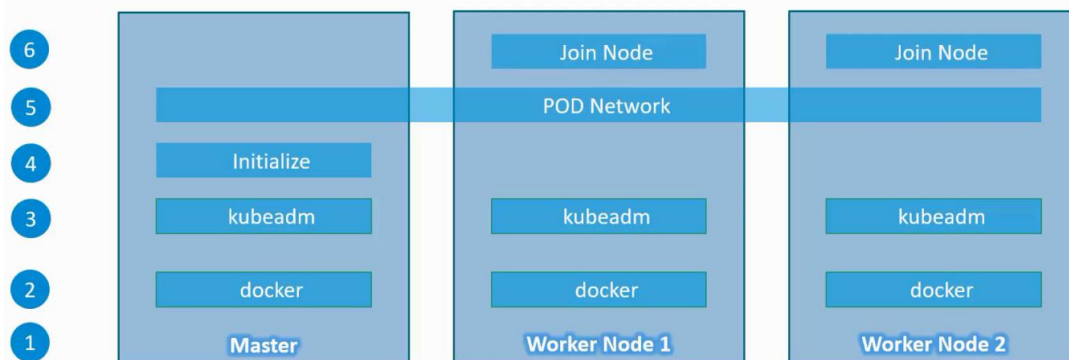
kubeadm help us bootstrapping the k8s cluster.

Steps



Initializing the master, during this process all required components are installed and configured on the master node.

Steps



Installation Steps

step1: In each node Install **pre-requisite**

docker

service docker status

service docker start

service docker stop

service docker restart

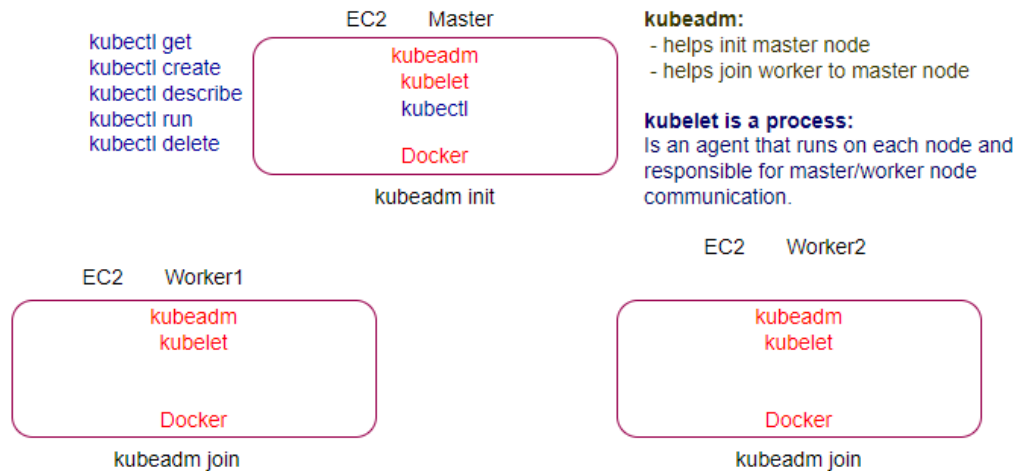
step2: In each node Install mandatory components

kubeadm: the command to bootstrap the cluster.

kubelet: the component that runs on all of the machines in your cluster and does things like starting pods and containers.

step2.1: In master node only

kubectl: the command line util to talk to your cluster.



step3: Elect the Master

In Master

`kubeadm init`

/*

`/etc/kubernetes/kubelet.conf` to point to a rotatable kubelet client certificate and key

[addons] Applied essential addon: CoreDNS

[addons] Applied essential addon: kube-proxy

Your Kubernetes control-plane has initialized successfully!

To start using your cluster, you need to run the following as a regular user:

`mkdir -p $HOME/. kube`

`sudo cp -i /etc/kubernetes/admin.conf $HOME/. kube/config`

`sudo chown $(id -u): $(id -g) $HOME/. kube/config`

*/

Now we have elected the master,

`kubectl get nodes`

```
root@masternode:~# kubectl get nodes/
The connection to the server localhost:8080 was refused - did you specify the right host or port?
root@masternode:~#
```

It means `kubectl` is unable to find the path of the api-Server, so not able to connect with api-server.

How to make it communicate to the api-server?

Note: config path? `/etc/kubernetes/admin.conf`

In order to work with kubectl you need to give the api-server address to the kubectl, hey! kubectl use the configuration which is available here
kubectl get nodes --kubeconfig=/etc/kubernetes/admin.conf

```
root@controlplane:~# kubectl get nodes --kubeconfig=/etc/kubernetes/admin.conf
NAME                STATUS    ROLES    AGE     VERSION
controlplane        NotReady  control-plane,master  6m43s   v1.20.2
```

It's not a best practice that we have to provide the configuration path to kubectl.

What is the best practice?

In java if we set the environment variable in classpath we can run the java program from anywhere

/*

To start using your cluster, you need to run the following as a regular user:

mkdir -p \$HOME/.kube // create a .kube folder in home directory

sudo cp -i /etc/kubernetes/admin.conf \$HOME/.kube/config

sudo chown \$(id -u):\$(id -g) \$HOME/.kube/config

*/

kubectl get nodes

```
root@masternode:~# kubectl get nodes
NAME                STATUS    ROLES    AGE     VERSION
masternode          NotReady  master   8m       v1.19.0
root@masternode:~#
```

Still NotReady Status

step4: Add networking solution (CNI) weave

Why we need to add Networking Solution?

When we run docker container, we don't need any extra network solution, you don't need to provide IP addresses to container because docker follows the CNM (Container Network Modal), Network features are in-build with docker.

In K8s does not give Networking by default, you have to install your own Networking solution K8s follows CNI (Container Network Interface).

Why K8s don't provide Networking Solution?

There is no standard container runtime you used in K8s, can use docker, rocket, and may not be compatible with all container run time.

Networks are just to provide Ip address to the pod and provide communication between the pods inside the cluster. Networking solution is for Pod IP, not for node IP.

Networking Solution CNI plugin: Weave (to establish a communication between different containers in cluster)

/*

kubectl apply -f "https://cloud.weave.works/k8s/net?k8s-version=\$(kubectl version | base64 | tr -d '\n')"

***/**

kubectl get nodes

```
root@controlplane:~# kubectl get nodes
NAME          STATUS    ROLES    AGE   VERSION
controlplane  Ready     control-plane,master  17m   v1.20.2
```

step4.1: create token

```
root@controlplane:~# kubeadm token create --print-join-command
kubeadm join 10.150.0.24:6443 --token furi68.3gklwpu62eljri22 --discovery-token-ca-cert-hash sha256:993ed89a946d5d127cb38033af05713ceaa1a9ad48b435105a08ab585e245f98
root@controlplane:~#
```

step5: Add the worker to the cluster

kubeadm join....

```
root@controlplane:~# kubectl get pods -A -o wide
NAMESPACE  NAME                                READY  STATUS   RESTARTS  AGE   IP            NODE          NOMINATED NODE  READINESS GATES
kube-system  coredns-74ff55c5b-mgjwz            1/1    Running  0          28m   10.32.0.2     controlplane  <none>          <none>
kube-system  coredns-74ff55c5b-rjsd8            1/1    Running  0          28m   10.32.0.3     controlplane  <none>          <none>
kube-system  etcd-controlplane                  1/1    Running  0          28m   10.150.0.24   controlplane  <none>          <none>
kube-system  kube-apiserver-controlplane         1/1    Running  0          28m   10.150.0.24   controlplane  <none>          <none>
kube-system  kube-controller-manager-controlplane 1/1    Running  0          28m   10.150.0.24   controlplane  <none>          <none>
kube-system  kube-proxy-6r9mf                   1/1    Running  0          117s   10.150.0.26   wnode02       <none>          <none>
kube-system  kube-proxy-f2ddd                   1/1    Running  0          2m18s   10.150.0.25   wnode01       <none>          <none>
kube-system  kube-proxy-mgtzb                   1/1    Running  0          28m   10.150.0.24   controlplane  <none>          <none>
kube-system  kube-scheduler-controlplane         1/1    Running  0          28m   10.150.0.24   controlplane  <none>          <none>
kube-system  weave-net-tf7bs                    2/2    Running  1          117s   10.150.0.26   wnode02       <none>          <none>
kube-system  weave-net-vjcwv                    2/2    Running  1          2m18s   10.150.0.25   wnode01       <none>          <none>
kube-system  weave-net-xln9q                    2/2    Running  1          12m   10.150.0.24   controlplane  <none>          <none>
```

kubelet is missing?

kubelet does not run as a container it run as a service.

service kubelet status

service kubelet start

service kubelet stop

service kubelet restart

/*

master components -> static pod managed by kubelet

kube-proxy & weave components available on all nodes -> DaemonSet

coredns (provides DNS service)-> HA access through the svc

***/**

```

root@controlplane:/etc/kubernetes# ls
admin.conf  controller-manager.conf  kubelet.conf  manifests  pki  scheduler.conf
root@controlplane:/etc/kubernetes# cd pki
root@controlplane:/etc/kubernetes/pki# ls
apiserver.crt  apiserver.key  ca.crt  front-proxy-ca.crt  front-proxy-client.key
apiserver-etcd-client.crt  apiserver-kubelet-client.crt  ca.key  front-proxy-ca.key  sa.key
apiserver-etcd-client.key  apiserver-kubelet-client.key  etcd  front-proxy-client.crt  sa.pub
root@controlplane:/etc/kubernetes/pki# cd ..
root@controlplane:/etc/kubernetes# cd manifests/
root@controlplane:/etc/kubernetes/manifests# ls
etcd.yaml  kube-apiserver.yaml  kube-controller-manager.yaml  kube-scheduler.yaml
root@controlplane:/etc/kubernetes/manifests#

```

kubelet. yaml and certificates are missing?

```

root@controlplane:/var/lib# cd kubelet/
root@controlplane:/var/lib/kubelet# ls
config.yaml  device-plugins  pki  plugins_registry  pods
cpu_manager_state  kubeadm-flags.env  plugins  pod-resources
root@controlplane:/var/lib/kubelet# cd pki/
root@controlplane:/var/lib/kubelet/pki# ls
kubelet-client-2020-12-03-16-49-02.pem  kubelet-client-current.pem  kubelet.crt  kubelet.key
root@controlplane:/var/lib/kubelet/pki#

```

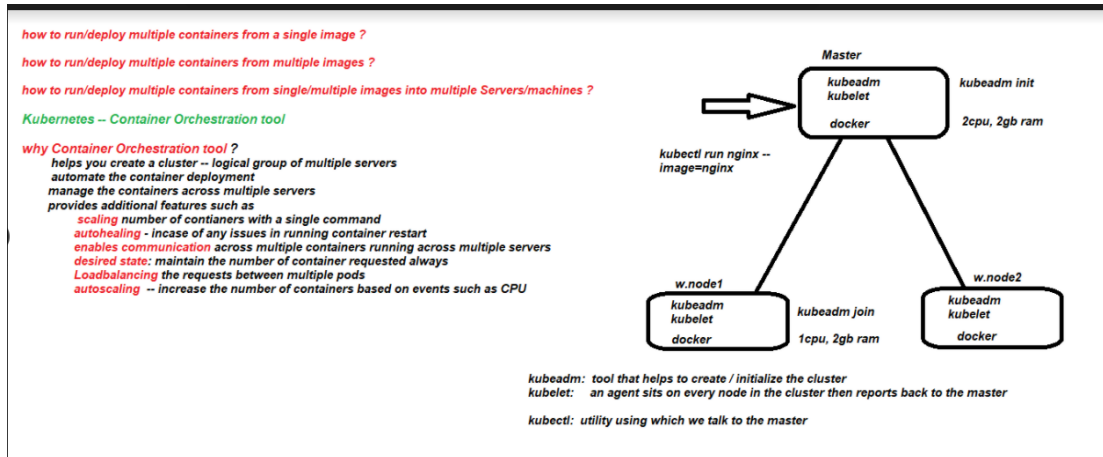
/var/lib/kubelet // config. yaml

/var/lib/kubelet/pki // certificates

Control-plane node(s)

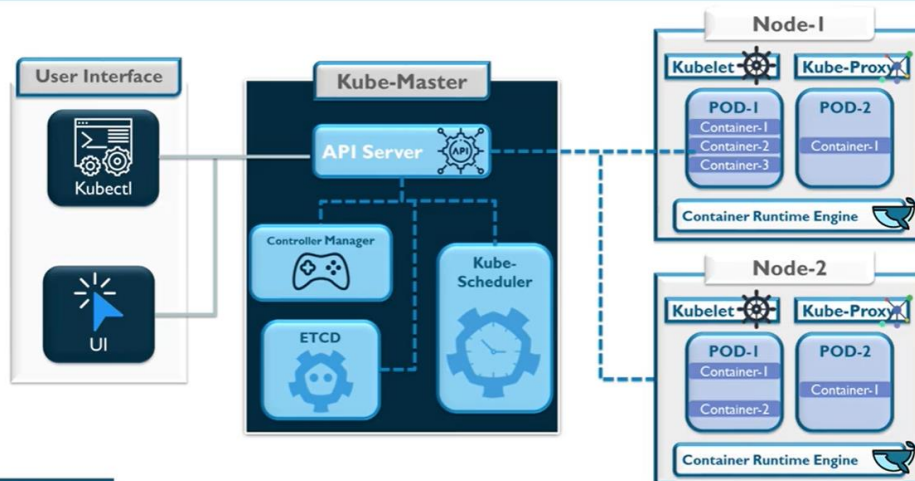
Protocol	Direction	Port Range	Purpose	Used By
TCP	Inbound	6443*	Kubernetes API server	All
TCP	Inbound	2379-2380	etcd server client API	kube-apiserver, etcd
TCP	Inbound	10250	Kubelet API	Self, Control plane
TCP	Inbound	10251	kube-scheduler	Self
TCP	Inbound	10252	kube-controller-manager	Self

K8s Architecture

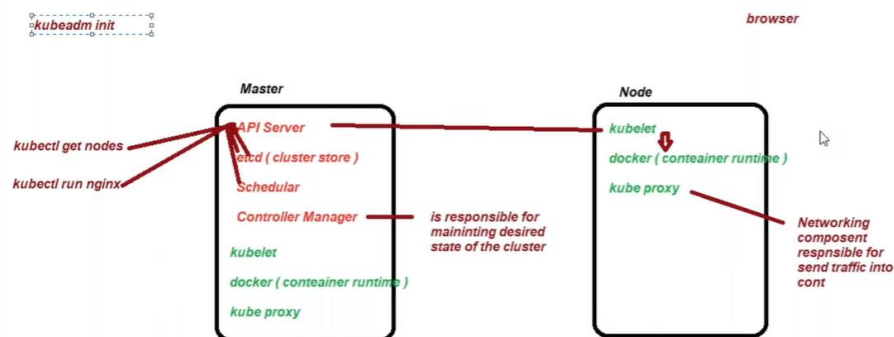


The purpose of K8s is to host your application in form of containers in an automated fashion so that you can easily deploy as many instances of your application as many they required and easily enable a communication between different services within your application.

Kubernetes Architecture



Utility and Components



K8s is a container management tool not a containerization tool (docker)

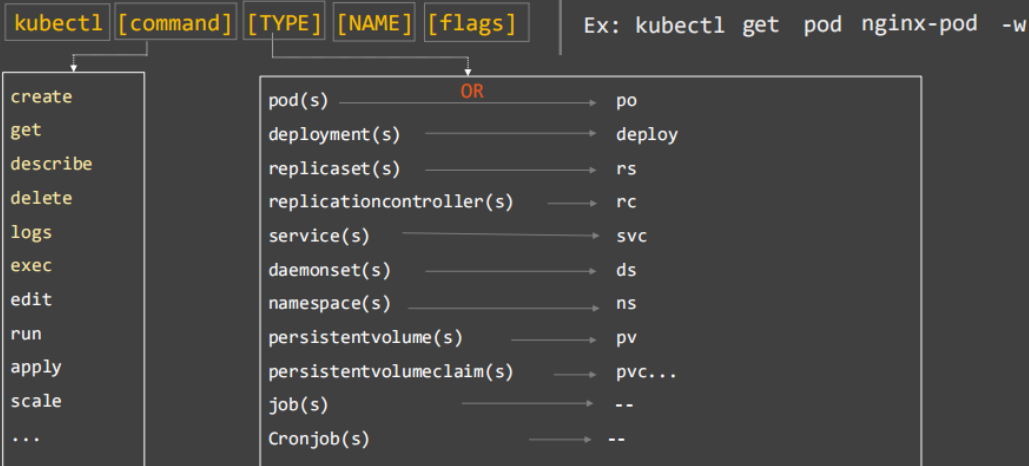
Utilities:

kubectl: Is a command line utility for running commands against kubernetes cluster interact with **K8s master(api-Server)** for creating/ updating/ deleting/ describe the resources in the cluster.

kubectl syntax: kubectl [command] [type] [name] [flags]

kubectl is the client utility run on only the master

Kubectl: Syntax



Kubectl - create, get

Kubectl create -f example.yaml Create a resource from a file

```
$ kubectl create -f pod-example.yaml
```

```
$ kubectl create -f deploy-example.yaml
```

```
$ kubectl create -f <directory>
```

Kubectl get [TYPE(s)] [NAME(s)] [FLAGS]- List one or more resources

```
$ kubectl get pods <pod-name>
```

```
$ kubectl get pods -o wide
```

```
$ kubectl get pods,deploy
```

Kubectl - describe, delete

kubectl describe [TYPE] [NAME] - Display detailed state of one or more resources

```
$ kubectl describe nodes <node-name>
```

```
$ kubectl describe pods <pod-name>
```

```
$ kubectl describe pods
```

kubectl delete - Delete resources

```
$ kubectl delete -f pod.yaml
```

```
$ kubectl delete pods,services -l name=<label-name>
```

```
$ kubectl delete pods --all
```

Kubectl - exec, logs

kubectl exec - Execute a command against a container in a pod.

```
$ kubectl exec <pod-name> date
```

```
$ kubectl exec <pod-name> -c <container-name> date
```

```
$ kubectl exec -ti <pod-name> /bin/bash
```

kubectl logs - Print the logs for a container in a pod.

```
$ kubectl logs <pod-name>
```

```
$ kubectl logs -f <pod-name>
```

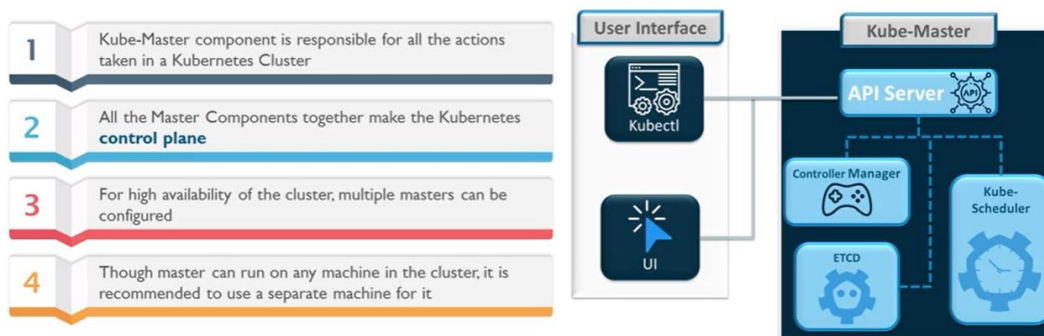
kubeadm: (bootstrap)

Using kubeadm utility, which helps us to initialize the cluster, everything deployed as a Pod

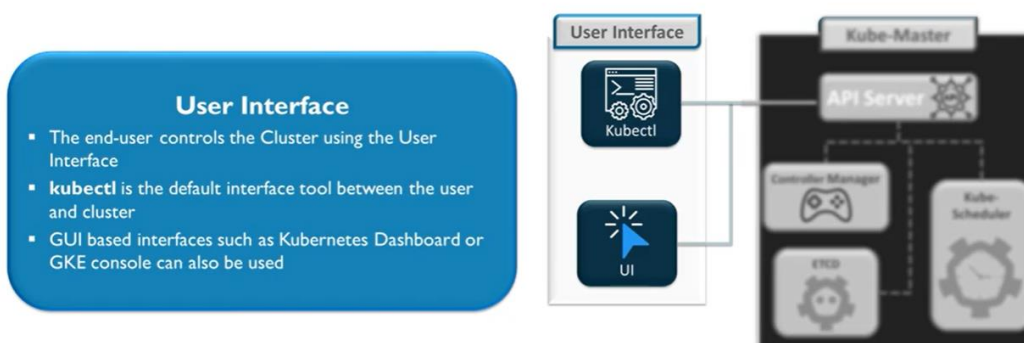
Command	Description
kubeadm init	to bootstrap a Kubernetes control-plane node K8S master also called control-plane

kubeadm join	to bootstrap a Kubernetes worker node and join it to the cluster
kubeadm upgrade	to upgrade a Kubernetes cluster to a newer version
kubeadm token	to manage tokens for kubeadm join
kubeadm reset	to revert any changes made to this host by kubeadm init or kubeadm join
kubeadm version	to print the kubeadm version

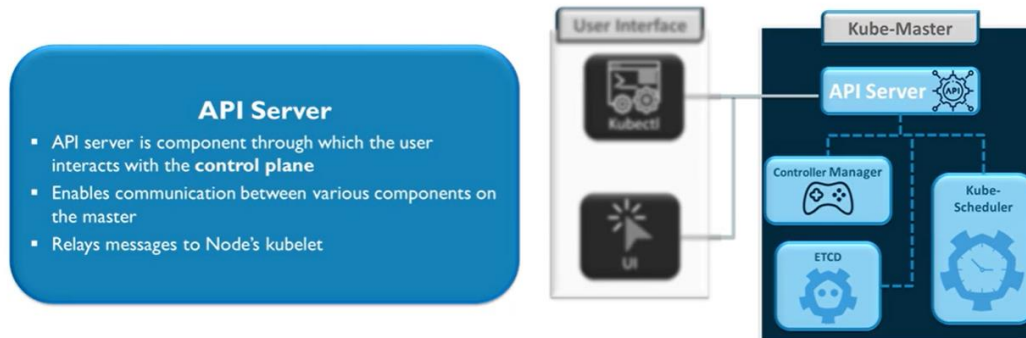
Kubernetes Architecture: Master Component



Master Component: User Interface



Master Component: API-Server



kube-apiserver is primary management component in k8s, when you run kubectl command, the kubectl utility in fact reaches to the kube-apiserver, kube-apiserver first authenticates the request and validates it and then retrieves the data from etcd cluster and responds back with the requested information.

`cat /etc/kubernetes/manifests/kube-apiserver.yaml`

Master Component: ETCD



Entire cluster meta-data managed by etcd, store entire information about your cluster such as information about the following.

- Nodes
- Pods
- Config
- Secrets
- Accounts

- Roles
- Bindings
- Which CNI is used?

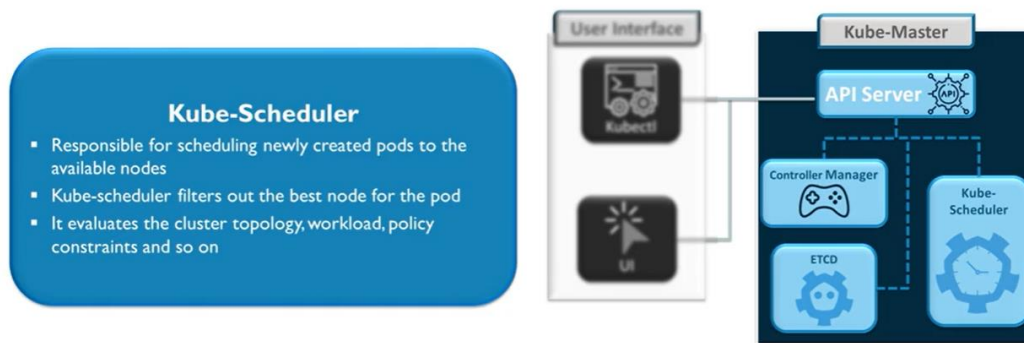
When api-server receives a request for **get Object**, it checks with etcd to send response back.

Once receives the Pod creation confirmation from kubelet, first stores the information in etcd and then sends response back.

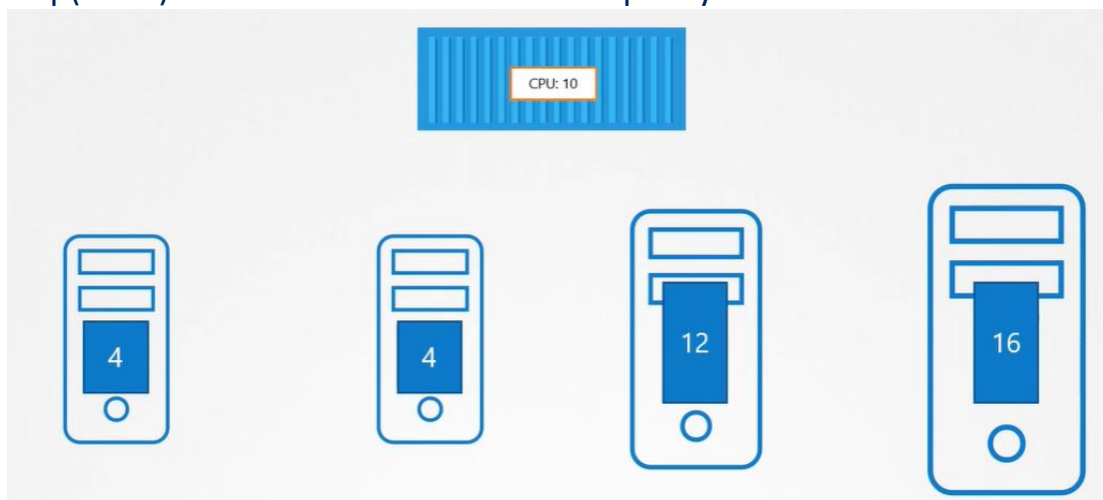
ETCDCTL is the CLI tool used to interact with ETCD.

cat /etc/kubernetes/manifests/etcd.yaml

Master Component: Kube-Scheduler



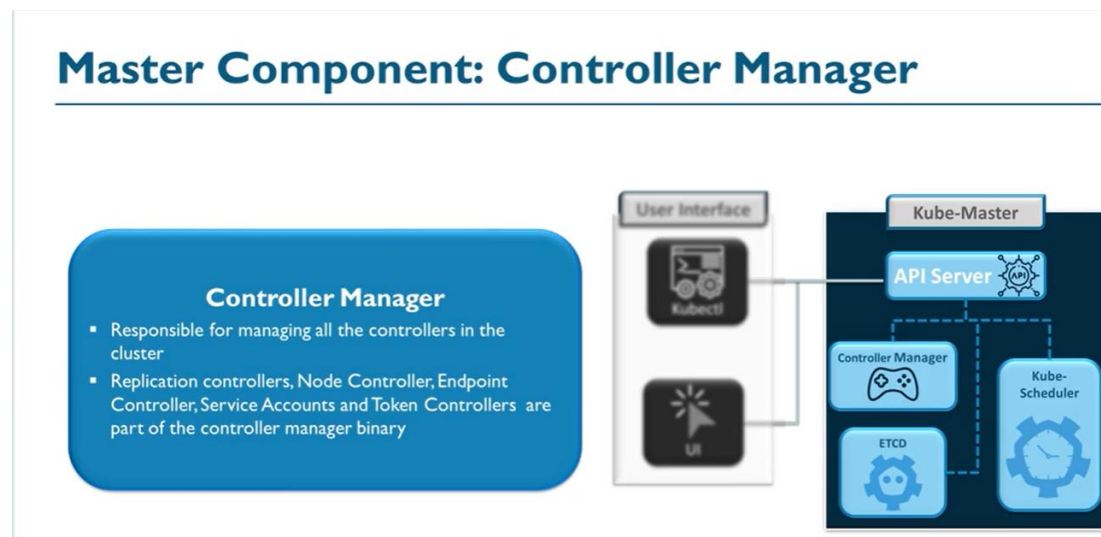
Is responsible for scheduling pods on the nodes. it is responsible only for deciding which pod goes on which node it doesn't actually place the Pod on the nodes that's the Job of the kubelet, to ensure right container(pod) on the right ship(node). node must have sufficient capacity to accommodate those containers.



When api-server receives a request for Object creation, it sends the request to scheduler, asks to scheduler hey! Scheduler can you please find a node where I can create a container/object.

Scheduler duty is to find appropriate node for deploy the container, and sends the response back to api-server, hey! Api-server please deploy the container at node1. It assign appropriate node for a pod based on the resource requirement of Pod and available resources in Node. It keeps track of resource utilization and makes sure that workload is not scheduled on nodes which are already full. If there are no suitable nodes, the pods are put in a **pending state** until such a node appears.

`cat /etc/kubernetes/manifests/kube-scheduler.yaml`



Node Controller: Is responsible for on-boarding new nodes on the cluster handle situation where node is unavailable or get destroyed.

Is responsible for monitoring the status of the nodes and takes necessary actions to keeps the application running. It does through the kube-apiserver, Node controller checks the status of nodes in every **5 sec**, It checks the heartbeats of the node, if it stops receiving the heartbeat from the node then the node is mark as UNREACHABLE but it waits for 40 sec before marking it UNREACHABLE after the node mark as UNREACHABLE it gives 5 min to come back up if it doesn't, it remove the Pod assigned to this node and provision them in healthy one if the Pod is a part of rs.

- **Watch status**
- **Remediate Situation**
- **Node Monitor Period =5s**
- **Node Monitor Grace Period=40s**

- **POD Eviction Timeout = 5m**

Replication Controller: Ensures that desired number of containers running all time.

- **Auto healing**

Cloud-controller-manager Specific to Cloud.

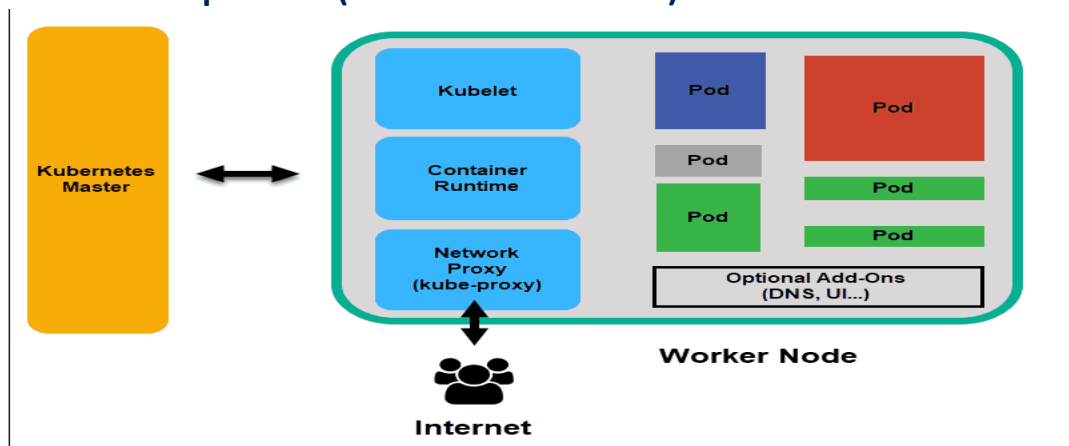
They all grouped together in kube-controller manager package.

Monitoring and maintain the state of the cluster, current vs. desired state.

if you are running 3 replica then its k8s responsibility that if any replica down then kubelet starts reporting to api-server , saying one of the container is down can you please check it, api-server requesting to controller-manager go and validate and then it compares with current and desired state current state is 2 and desired state is 3, then it convey to api-server, controller-manager continue looking in to current and desired state in cluster, every 5 sec, continues checking the resources state in cluster.

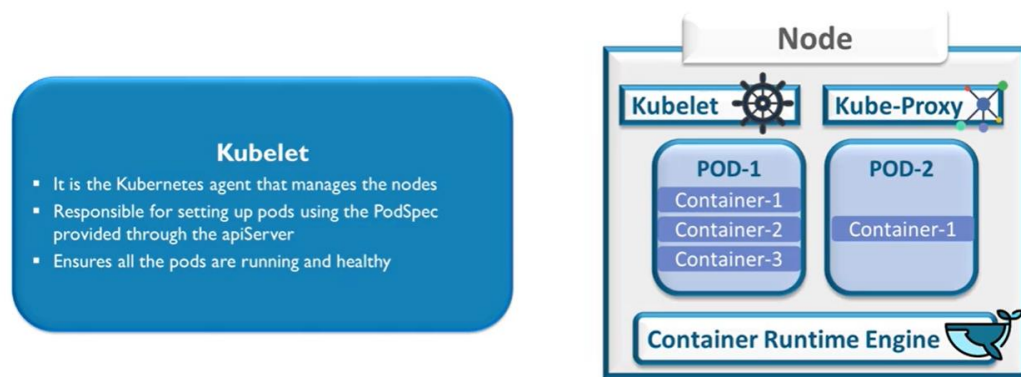
```
cat /etc/kubernetes/manifests/kube-controller-manager.yaml
ps -aux | grep kube-controller-manager
```

Worker Components (Actual runs the Pods)



Worker nodes listen to the API Server for new work assignments; they execute the work assignments and then report the results back to the API Server in Master node.

Node Component: Kubelet



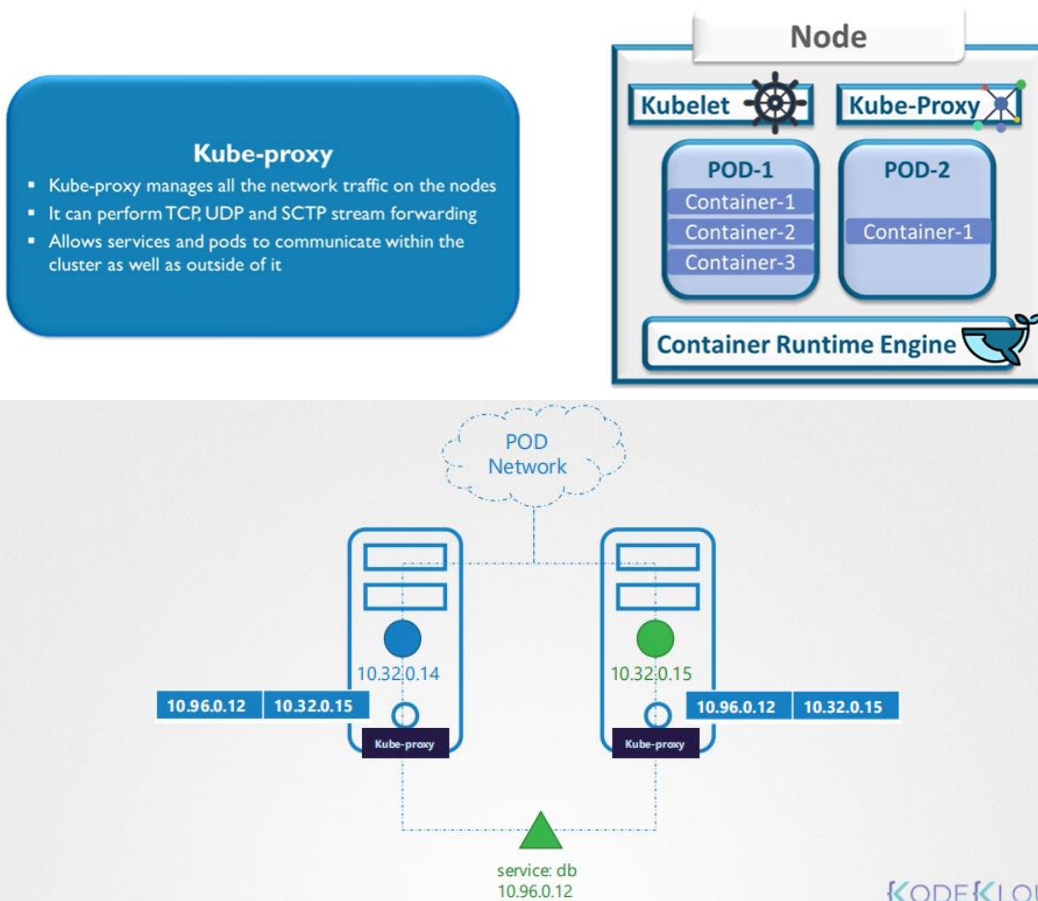
kubelet run as a service on all nodes

service kubelet status

// on all nodes

- Agent service running on each worker node.
- Is the only component interacting with the apiserver?
- After getting information from scheduler to deploy pod on node1, api-server interacts with kubelet of node1 and convey to him please create a Pod, now its responsibility of kubelet to create a Pod in that specific node. Kubelet Interact with underlying container run time engine (docker) to create and manage pods, get specification of pod from apiServer. Then docker creates a Pod internally (Interact with docker registry to download image) and inform to kubelet, kubelet pass the information back to api-server.
- Once Pod is created interacts with underline CNI to assign IP to the Pod.
- Making ensure that container is running.
- provide metrics and health check information (after 5 sec) to apiserver (cAdvisor), which api-stores in etcd
- Provide heart beats to apiserver, acts as an agent on worker nodes, which api-server stores in etcd.

Node Component: Kube-proxy



kube-proxy is a process that runs on each node in the k8s cluster, its job is to look for new services and every time a new service is created it creates the appropriate rules (DNS) on each node to forward request to those services to the backend application. **kubeadm** define it as a **daemonset controller**.

- Its works at service management layer.
- Service is a layer of abstraction between pod and requester
- It's a packet forwarding component between service and pod.

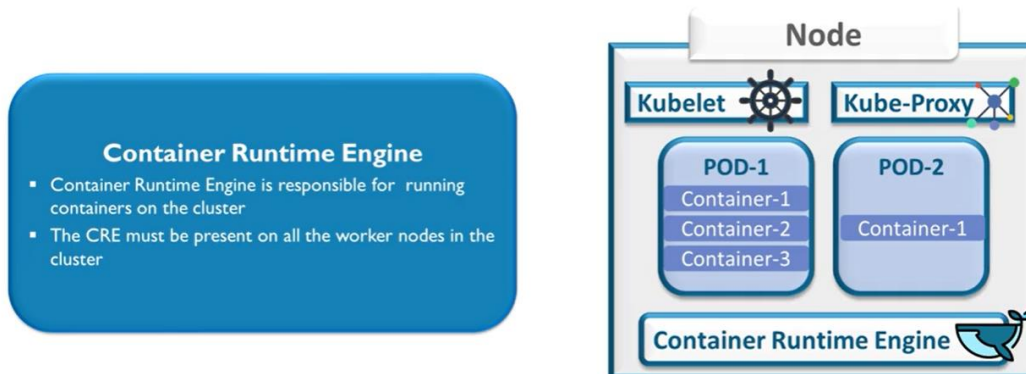
It's a kubelet inform node status to api-server, sometimes it may happen that kubelet is down, in that case node's status is shown as NotReady.

- service kubelet status
- service docker status

kubectl describe node worker1

Conditions:						
Type	Status	LastHeartbeatTime	LastTransitionTime	Reason	Message	
NetworkUnavailable	False	Fri, 04 Dec 2020 05:05:41 +0000	Fri, 04 Dec 2020 05:05:41 +0000	WeaveIsUp	Weave pod has set this	
MemoryPressure	False	Fri, 04 Dec 2020 06:50:13 +0000	Thu, 03 Dec 2020 17:51:21 +0000	KubeletHasSufficientMemory	kubelet has sufficient	
memory available						
DiskPressure	False	Fri, 04 Dec 2020 06:50:13 +0000	Thu, 03 Dec 2020 17:51:21 +0000	KubeletHasNoDiskPressure	kubelet has no disk pre	
Pressure						
PIDPressure	False	Fri, 04 Dec 2020 06:50:13 +0000	Thu, 03 Dec 2020 17:51:21 +0000	KubeletHasSufficientPID	kubelet has sufficient	
PID available						
Ready	True	Fri, 04 Dec 2020 06:50:13 +0000	Thu, 03 Dec 2020 17:52:02 +0000	KubeletReady	kubelet is posting read	
y status. AppArmor enabled						
Addresses:						

Node Component: Container Runtime Engine



The container runtime pulls images from a container image registry and starts and stops containers. A 3rd party software or plugin, such as Docker, usually performs this function.